

conditions, the resulting process is given the new security context. When there are no such rules, the new process inherits the security context of its parents.

To check all type transition rules that apply to processes (classes of type process), issue the following commands:

```
[root@vbg-work ~]# search -T -c process
type.transition user_xinetd_t fusemount_exec.t : process mount.t;
type.transition staff_exec.t nlogin_config_exec.t : process nlogin_config.t;
type.transition isnsd.t fusemount_exec.t : process mount.t;
type.transition xend.t fusemount_exec.t : process mount.t;
type.transition sysadm_java.t fusemount_exec.t : process mount.t;
type.transition rfc63_rsdservice.t fail2ban_initrc_exec.t : process initrc.t;
type.transition init_t afs_bosserver_exec.t : process afs_bosserver.t;
type.transition hotplug_t elcd_initrc_exec.t : process initrc.t;
type.transition rfc63_rsdservice.t ssad_initrc_exec.t : process initrc.t;
type.transition init_t apmd_exec.t : process apmd.t;
type.transition qmail_local.t qmail_queue_exec.t : process qmail_queue.t;
type.transition staff_exec.t gpg_agent_exec.t : process gpg_agent.t;
...
```

Based on the above rules, the syntax of type transition rules that apply to processes can be understood as:

```
type.transition [Parent Process Type] [Type of File being executed
to create a process] : process [Type of The New Process created]
```

To check the above rules, let us look at the example of the Web server process discussed earlier.

When executing the executable `/usr/sbin/httpd` directly to create the `httpd` process:

- The parent process is the shell (because we are executing the command `/usr/sbin/httpd` under the Bash shell).
- The type of the parent process is `unconfined_t`. (Execute the following command to check.)  

```
[root@vbg-work ~]# ps axZ | grep bash
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 2400 pts/0 Ss  0:00 bash
```
- The type of the file being executed is `httpd_exec_t`. (Execute the following command to check.)  

```
[root@vbg-work ~]# ls -lZ /usr/sbin/httpd
-rwxr-xr-x. root root system_u:object_r:httpd_exec_t:s0 /usr/sbin/httpd
```

Let us check in our policy if such a rule exists:

```
[root@vbg-work ~]# search -T -c process | grep httpd_exec.t | grep unconfined_t
[root@vbg-work ~]#
```

This shows that we do not have a type transition rule to set the security context of new processes created by executing files of type `httpd_exec_t` under a parent process of type `unconfined_t`. In such a scenario, i.e., in the absence of any type transition rule, the new process takes the type of the parent; therefore, the `httpd` process inherits its security context from Bash (which is `unconfined_t`).

Let us now analyse the other case. When starting the `httpd` service through initialisation scripts:

- The parent process is the shell (because we are executing the script `/etc/init.d/httpd` under Bash).
- The type of the parent process is `unconfined_t`. (Execute the following command to check.)

```
[root@vbg-work ~]# ps axZ | grep bash
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 2400 pts/0 Ss  0:00 bash
```

- The type of the file being executed is `httpd_initrc_exec_t`. (Execute the following command to check.)

```
[root@vbg-work ~]# ls -lZ /etc/init.d/httpd
-rwxr-xr-x. root root system_u:object_r:httpd_initrc_exec_t:s0 /etc/init.d/httpd
```

Let us check in our policy if such a rule exists:

```
[root@vbg-work ~]# search -T -c process | grep \
httpd_initrc_exec.t | grep unconfined_t
type.transition unconfined_t httpd_initrc_exec.t : process initrc.t;
[root@vbg-work ~]#
```

Finally, we have found a match. This rule states that if a file of type `httpd_initrc_exec_t` is executed under a parent process of type `unconfined_t`, the resulting process should get a type of `initrc_t`. This is good, but we had observed that the Web server process had a type of `httpd_t` and this rule states that the type of the resulting process is `initrc_t`. Is there an error? Is there something missing?

There is, indeed, something missing. It is important to note that the Web server process is created on executing the binary file (`/usr/bin/httpd`). Therefore, the init script actually executes this file.

But what happens is that the shell script process created by executing the initialisation script (`/etc/init.d/httpd`) is now running with a process security context bearing the type `initrc_t` (as per the rule above). When this process further executes the Web server binary (`/usr/sbin/httpd`), a new context is created. Therefore, there is another scenario that comes into play here, which is:

- The parent process is the shell script process created by executing the script `/etc/init.d/httpd`.
- The type of the parent process is `initrc_t` (because this is the type of process created by executing the init script from under Bash, as per the type transition rule above).
- The type of the file being executed is `httpd_exec_t` (as the shell script further executes the binary `/usr/sbin/httpd`). Execute the following command to check:

```
[root@vbg-work ~]# ls -lZ /usr/sbin/httpd
-rwxr-xr-x. root root system_u:object_r:httpd_exec_t:s0 /usr/sbin/httpd
```

Let us see if we have a type transition rule in the policy for the above:

```
[root@vbg-work ~]# search -T -c process | grep httpd_exec.t | grep initrc_t
type.transition initrc_t httpd_exec.t : process httpd.t;
[root@vbg-work ~]#
```